

Question No:8

Question description

You are given an array A of length N which is initialised with 0 . You will be given Q queries of two types:

- 1 k : set value 1 at index k in array A
- 2 y : print the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Note: Indexing is 1 based

Constraints

- $1 \leq n \leq 10^9$
- $1 \leq q \leq 5 * 10^5$
- $1 \leq y, k \leq n$

Input Format

First line contains two integers N and Q separated by a space.
The next Q lines contain the type of query (i.e. either a 1 or a 2), then a space, then for type 1 queries integer k and for type 2 queries integer y

Output Format

For each query type 2 , print in new line, the smallest index x which is greater than or equal to y and having value 1 . If there is no such index print -1 .

Explanation for Test case 1

- For first query: $2\ 3$, there is no index greater than or equal index 3 , having value -1 , so the answer is -1 .
- For second query: $1\ 2$, set value -1 at index 2 .
- For third query: $2\ 1$, index 2 is greater than index 1 , having value -1 , so the answer is 2 .
- For fourth query: $2\ 3$, there is no index greater than or equal index 3 , having value -1 , so the answer is -1 .
- For fifth query: $2\ 2$, index 2 is equal to index 2 , having value -1 , so the answer is 2 .

Logical Test Cases

Test Case 1

INPUT (STDIN)

5 5
2 3
1 2
2 1
2 3
2 2

EXPECTED OUTPUT

-1
2
-1
2

Test Case 2

INPUT (STDIN)

5 5
2 1
1 2
2 2
1 2
2 3

EXPECTED OUTPUT

-1
2
-1

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXQ 500005
5
6  typedef struct {
7      int type;
8      int value;
9  } Query;
10
11  Query q[MAXQ];
12  int pos[MAXQ];
13  int tree[4 * MAXQ];
14
15  int compare(const void *a, const void *b) {
16      return (*(int *)a - *(int *)b);
17  }
18
19  // Find the first active index at or after start
20  int findFirst(int node, int left, int right, int start) {
21      if (right <= start || tree[node] == 0) {
22          return -1;
23      }
24
25      if (right - left == 1) {
26          return left;
27      }
28
29      int mid = (left + right) / 2;
```

```

29     int mid = (left + right) / 2;
30
31     int result = findFirst(node * 2, left, mid, start);
32
33     if (result != -1) {
34         return result;
35     }
36
37     return findFirst(node * 2 + 1, mid, right, start);
38 }
39
40 // Activate an index in the segment tree
41 void update(int node, int left, int right, int index) {
42     if (right - left == 1) {
43         tree[node] = 1;
44         return;
45     }
46
47     int mid = (left + right) / 2;
48
49     if (index < mid) {
50         update(node * 2, left, mid, index);
51     } else {
52         update(node * 2 + 1, mid, right, index);
53     }
54
55     tree[node] = tree[node * 2] + tree[node * 2 + 1];

```

```
56     }
57
58     // Find the first compressed position >= y
59     int lowerBound(int arr[], int n, int y) {
60         int left = 0;
61         int right = n;
62
63         while (left < right) {
64             int mid = left + (right - left) / 2;
65
66             if (arr[mid] < y) {
67                 left = mid + 1;
68             } else {
69                 right = mid;
70             }
71         }
72
73         return left;
74     }
75
76     int main() {
77         int N, Q;
78         scanf("%d %d", &N, &Q);
79
80         int count = 0;
```

```

81
82 // Read all queries and collect update indices
83 for (int i = 0; i < Q; i++) {
84     scanf("%d %d", &q[i].type, &q[i].value);
85
86     if (q[i].type == 1) {
87         pos[count++] = q[i].value;
88     }
89 }
90
91 // Sort update indices
92 qsort(pos, count, sizeof(int), compare);
93
94 // Remove duplicate indices
95 int size = 0;
96
97 for (int i = 0; i < count; i++) {
98     if (i == 0 || pos[i] != pos[i - 1]) {
99         pos[size++] = pos[i];
100     }
101 }
102
103 // Process queries in their original order
104 for (int i = 0; i < Q; i++) {
105     int value = q[i].value;
106
107     if (q[i].type == 1) {
108         int index = lowerBound(pos, size, value);
109
110         update(1, 0, size, index);
111     } else {
112         int start = lowerBound(pos, size, value);
113
114         int result = findFirst(1, 0, size, start);
115
116         if (result == -1) {
117             printf("-1\n");
118         } else {
119             printf("%d\n", pos[result]);
120         }
121     }
122 }
123
124 return 0;
125 }

```

Output:

Test Case-1:

```
5 5
2 3
1 2
2 1
2 3
2 2
-1
2
-1
2
PS C:\Users\kesha\OneDri
```

Test Case-2:

```
5 5
2 1
1 2
2 2
1 2
2 3
-1
2
-1
PS C:\Users\kesha\OneDri
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236